

Manual Técnico de Uso

Motor de mensageria distribuído com Event Sourcing e CQRS

Guia completo para instalar, configurar, operar e integrar o Brado: broker TCP, banco de eventos, modelo de leitura, outbox, saga e painel de monitoramento.

Brado · Distributed-Event-Messaging-Engine

Manual técnico · versão 1.0 · outubro de 2026

Copyright (©) 2026 Clara Portes

Sumário

01	Introdução	o que é e como se organiza
02	Instalação e execução	SQLite, PostgreSQL e Docker
03	Configuração	todas as chaves do appsettings
04	Conceitos do broker	tópicos, offsets, grupos, DLQ
05	Protocolo TCP	layout binário e sequências
06	Cliente .NET	publicar e consumir
07	API HTTP	comandos, consultas e eventos
08	Painel de monitoramento	o que cada área mostra
09	Event Sourcing e CQRS	agregado, snapshots, projeções
10	Saga de atendimento	fluxo e compensação
11	Outbox e garantias de entrega	at-least-once sem perda
12	Testes e benchmarks	como rodar e o que cobrem
13	Operação e solução de problemas	produção, backup, diagnóstico
14	Automação de commits	workflow de 5 execuções diárias

Copyright (©) 2026 Clara Portes

01 Introdução

O Brado combina um broker de mensagens com um banco de eventos em um único processo .NET 10, mantendo escrita e leitura separadas.

O sistema é dividido em projetos com responsabilidades estreitas. Cada um pode ser usado sozinho — o cliente TCP, por exemplo, não depende de nada além do protocolo.

Projeto	Responsabilidade
`Brado.Protocol`	Quadro binário, codec com `Span<T>`, leitor e escritor sobre `Stream`, payloads auxiliares e validação de nomes.
`Brado.Domain`	Agregado `Order`, eventos de domínio, snapshot e o envelope publicado no broker.
`Brado.Broker`	Tópicos, grupos de consumo, servidor TCP (`BrokerServer`), handlers por tipo de quadro e métricas.
`Brado.Client`	Cliente TCP com multiplexação por correlação, assinaturas e ACK automático.
`Brado.EventStore`	Event store, outbox, saga store, inbox de idempotência e journal do broker (Dapper + SQL).
`Brado.Cqrs`	Comandos, consultas, dispatchers, modelo de leitura EF Core e motor de projeções.
`Brado.Workers`	Outbox dispatcher, projection worker e saga como `BackgroundService`.
`Brado.Server`	Host ASP.NET Core: API, SSE, painel e simulador de tráfego.

Quando usar cada parte

Somente broker

Filas e pub/sub entre serviços: publique em tópicos e consuma em grupos com prefetch, ACK e DLQ.

Event Sourcing

Domínios que precisam de histórico completo e auditável, com reconstrução do estado em qualquer versão.

CQRS

Leituras rápidas em visões desnormalizadas, independentes do modelo de escrita, reconstruíveis por replay.

Outbox + Saga

Transações distribuídas sem two-phase commit: cada passo publica eventos com garantia e a saga compensa falhas.

Copyright (©) 2026 Clara Portes

02 Instalação e execução

Três formas de subir o Brado: SQLite local sem configuração, PostgreSQL local ou tudo em contêineres.

Requisitos

- .NET SDK 10 (o `global.json` fixa 10.0.100 com *roll forward* para versões mais novas).
- Docker para PostgreSQL e para os testes de integração (Testcontainers).
- Portas livres: `5080` (HTTP e painel) e `7420` (broker TCP).

SQLite (padrão)

```
git clone https://github.com/Violet-pixel/Distributed-Event-Messaging-Engine.git
cd Distributed-Event-Messaging-Engine
dotnet restore Brado.slnx
dotnet run --project src/Brado.Server
```

Os bancos ficam em `data/brado-events.db` (escrita) e `data/brado-read.db` (leitura). O schema é criado na inicialização, de forma idempotente.

PostgreSQL

```
docker compose up -d postgres
dotnet run --project src/Brado.Server --environment Postgres
```

O ambiente `Postgres` carrega `appsettings.Postgres.json`, que aponta o event store para o banco `brado` e o modelo de leitura para `brado_read` (criado automaticamente).

Docker Compose completo

```
docker compose up --build
```

Sobe PostgreSQL 17 e o servidor com tráfego ambiente ligado. Painel em `http://localhost:5080`.

> **Dica.** Abra `http://localhost:5080/?demo` para ver o painel em modo demonstração, com dados sintéticos, mesmo sem servidor rodando.

Copyright (©) 2026 Clara Portes

03 Configuração

Todas as opções ficam em appsettings.json e podem ser sobrescritas por variável de ambiente (Secao__Chave).

Broker

Chave	Padrão	Descrição
`Host` / `Port`	0.0.0.0 / 7420	Endereço de escuta do servidor TCP.
`MaxConnections`	2048	Conexões simultâneas; além disso o servidor responde ERROR e fecha.
`MaxPayloadBytes`	4 MiB	Tamanho máximo do payload; validado no cabeçalho antes de alocar memória.
`OutboundQueueCapacity`	8192	Capacidade do `Channel` de saída por conexão (backpressure).
`RetainedMessagesPerTopic`	100000	Janela de mensagens em memória e no journal por tópico.
`DefaultPrefetch`	64	Entregas sem ACK por membro quando o cliente não informa.
`AckTimeout`	00:00:30	Prazo para ACK; depois disso a mensagem é reentregue.
`MaxDeliveryAttempts`	5	Tentativas antes de enviar para `<tópico>.dlq`.
`DeadLetterSuffix`	.dlq	Sufixo do tópico de mensagens mortas.
`PersistMessages`	true	Liga o journal durável e a gravação de offsets dos grupos.

EventStore e ReadModel

Chave	Padrão	Descrição
`EventStore:Provider`	Sqlite	`Sqlite` ou `Postgres`.
`EventStore:ConnectionString`	Data Source=data/brado-events.db	Conexão do lado de escrita.
`EventStore:SnapshotEvery`	50	Grava snapshot a cada N eventos de um stream (0 desliga).
`ReadModel:Provider`	Sqlite	Banco do lado de leitura; pode ser diferente do de escrita.
`ReadModel:ConnectionString`	Data Source=data/brado-read.db	Conexão do modelo de leitura (EF Core).

Copyright (©) 2026 Clara Portes

Messaging, Outbox, Projections, Saga e Simulation

Chave	Padrão	Descrição
`Messaging:Mode`	InProcess	`InProcess` usa o broker do próprio processo; `Tcp` conecta em `BrokerHost:BrokerPort`.
`Outbox:BatchSize`	200	Mensagens reservadas por ciclo.
`Outbox:PollingInterval`	00:00:00.500	Espera máxima entre ciclos (o append acorda o worker antes).
`Outbox:Lease`	00:00:30	Tempo de reserva de um lote por worker.
`Outbox:BaseRetryDelay` / `MaxRetryDelay`	1 s / 5 min	Backoff exponencial com jitter de ±20%.
`Outbox:DispatchedRetention`	7 dias	Mensagens entregues mais antigas são removidas.
`Projections:BatchSize`	500	Eventos projetados por transação.
`Saga:Enabled` / `Topic`	true / orders.events	Liga a saga e define o tópico consumido.
`Saga:PaymentFailureRate` / `StockFailureRate`	0.08 / 0.10	Falhas dos serviços simulados.
`Simulation:Ambient`	false (true em Development)	Tráfego contínuo para demonstrações.

```
EventStore__Provider=Postgres
EventStore__ConnectionString="Host=db;Database=brado;Username=brado;Password=***"
Broker__MaxDeliveryAttempts=3
```

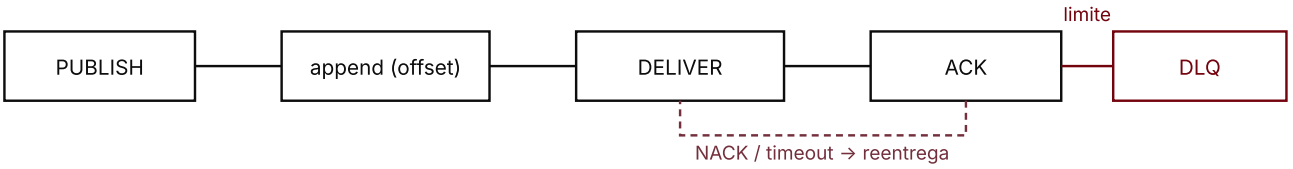

Copyright (©) 2026 Clara Portes

04 Conceitos do broker

O broker usa um log append-only por tópico e, sobre ele, grupos de consumo com semântica de fila.

Conceito	Como funciona no Brado
Tópico	Log com offsets crescentes a partir de 0. Criado no primeiro uso. Nomes aceitam letras, dígitos, `.`, `-` e `_` (até 200).
Retenção	Cada tópico mantém as últimas `RetainedMessagesPerTopic` mensagens; o descarte acontece em blocos de 10% para custo constante.
Grupo de consumo	Conjunto de membros que dividem as mensagens. Grupos diferentes recebem todas as mensagens do tópico de forma independente.
Membro	Uma assinatura TCP ou local. As entregas seguem round-robin entre membros com crédito.
Prefetch	Créditos por membro: quantas mensagens podem estar entregues e ainda sem ACK.
ACK	Confirma o offset e devolve o crédito. O offset confirmado do grupo é o menor offset ainda pendente (confirmação contígua).
NACK	Com `Requeue` a mensagem volta à fila; sem ele, ou após o limite de tentativas, vai para a DLQ.
Timeout	Entregas sem ACK após `AckTimeout` voltam para a fila na manutenção, que roda a cada segundo.
Posição inicial	`Latest` (−1) começa no próximo offset; `Earliest` (0) relê tudo que ainda está retido. Só vale para grupos novos.
Persistência	Com `PersistMessages`, mensagens vão para `broker_messages` em lotes e os offsets dos grupos para `consumer_offsets` a cada 5 s.

Ciclo de uma mensagem



Publicação, entrega, confirmação, reentrega e dead-letter.

Copyright (©) 2026 Clara Portes

05 Protocolo TCP

Quadros binários big-endian com cabeçalho fixo de 28 bytes, pensados para leitura sem cópia e validação antecipada.

Cabeçalho

Bytes	Campo	Tipo	Descrição
0–1	magic	u16	`0xA7E5` ; qualquer outro valor encerra a conexão.
2	versão	u8	`1` .
3	tipo	u8	Tipo do quadro (tabela abaixo).
4	flags	u8	`Requeue` = 1 (NACK), `Redelivered` = 2 (DELIVER).
5	reservado	u8	Zero.
6–13	correlação	i64	Casa requisição e resposta; em DELIVER/ACK/NACK é o id da assinatura.
14–21	offset	i64	Offset atribuído (PUBLISH_ACK) ou entregue (DELIVER, ACK, NACK).
22–23	tamTópico	u16	Bytes UTF-8 do tópico (até 512).
24–27	tamPayload	i32	Bytes do payload (até `MaxPayloadBytes`).

Tipos de quadro

Código	Tipo	Corpo
0x01 / 0x02	HELLO / HELLO_ACK	nome do cliente (UTF-8) / id da conexão (i64)
0x10 / 0x11	PUBLISH / PUBLISH_ACK	payload livre / offset no cabeçalho
0x20 / 0x21	SUBSCRIBE / SUBSCRIBE_ACK	`| tamGrupo u16 | grupo | início i64 | prefetch i32 |` / id da assinatura (i64)
0x22	UNSUBSCRIBE	correlação = id da assinatura
0x30	DELIVER	payload; correlação = assinatura; offset no cabeçalho
0x31 / 0x32	ACK / NACK	correlação = assinatura; offset; flag Requeue no NACK
0x40 / 0x41	PING / PONG	vazio
0x7F	ERROR	mensagem UTF-8

Copyright (©) 2026 Clara Portes

Sequência típica

```
cliente                                  broker
  HELLO(corr=1, "checkout-api")      ─►
                                     ◄─  HELLO_ACK(corr=1, conn=17)
  SUBSCRIBE(corr=2, pedidos, g, 0, 64) ─►
                                     ◄─  SUBSCRIBE_ACK(corr=2, sub=5)
                                     ◄─  DELIVER(sub=5, offset=0)
  ACK(sub=5, offset=0)               ─►
  PUBLISH(corr=3, pedidos, {...})    ─►
                                     ◄─  PUBLISH_ACK(corr=3, offset=1)
```

Ordem garantida. O SUBSCRIBE_ACK é enfileirado antes da primeira entrega, e o cliente registra a assinatura ao ler o ACK — nenhuma entrega chega para uma assinatura desconhecida.

Copyright (©) 2026 Clara Portes

06 Cliente .NET

BradoConnection encapsula conexão, correlação, keep-alive e consumo com ACK automático.

Conectar e publicar

```
await using var connection = await BradoConnection.ConnectAsync(new BradoClientOptions
{
    Host = "localhost",
    Port = 7420,
    ClientName = "checkout-api",
    RequestTimeout = TimeSpan.FromSeconds(10)
});

long offset = await connection.PublishAsync("pedidos.criados", "texto simples");
offset = await connection.PublishJsonAsync("pedidos.criados", new { pedido = 42, total = 359.30m
});
```

Consumir

```
await using var subscription = await connection.SubscribeAsync(
    topic: "pedidos.criados",
    group: "faturamento",
    handler: async (message, ct) =>
    {
        var dados = message.AsJson<PedidoCriado>();
        await faturamento.EmitirAsync(dados, ct);
    },
    prefetch: 64,
    startOffset: StartPosition.Earliest);
```

- O handler roda em sequência por assinatura; para paralelizar, abra mais assinaturas no mesmo grupo (em uma ou mais conexões).
- Retorno normal envia ACK; exceção envia NACK com requeue. `message.Redelivered` indica reentrega.
- Handlers devem ser idempotentes: a entrega é *at-least-once*.
- `PingAsync()` mede a latência; o keep-alive roda sozinho a cada 15 s.

Copyright (©) 2026 Clara Portes

07 API HTTP

Minimal APIs que expõem o lado de escrita (comandos), o de leitura (consultas), o event store e o broker.

Método	Rota	Descrição
POST	`/api/orders`	Cria pedido. Corpo: `customerRef`, `items[]`, `currency`, `submit`.
POST	`/api/orders/{{id}}/items`	Adiciona item (`sku` , `quantity` , `unitPrice`).
POST	`/api/orders/{{id}}/submit`	Envia o pedido e inicia a saga.
POST	`/api/orders/{{id}}/cancel`	Cancela (exige estorno se já pago).
GET	`/api/orders`	Lista paginada do modelo de leitura (`status` , `page` , `pageSize`).
GET	`/api/orders/{{id}}`	Detalhe com itens.
GET	`/api/sales/daily`	Vendas e estornos por dia.
GET	`/api/read-model`	Totais por status, receita e checkpoint.
POST	`/api/projections/rebuild`	Apaga o modelo de leitura e reconstrói por replay.
GET	`/api/streams`	Streams recentes (`search` , `skip` , `take`).
GET	`/api/streams/{{id}}/replay`	Eventos do stream com o estado após cada versão (`toVersion` opcional).
GET	`/api/events`	Log global a partir de `after` .
GET	`/api/broker`	Fotografia: métricas, tópicos, grupos e conexões.
POST	`/api/broker/topics/{{t}}/publish`	Publica o corpo bruto da requisição.
GET	`/api/stream`	Server-Sent Events (evento `snapshot` a cada segundo).
POST	`/api/simulate`	Gera pedidos e mensagens (`orders` , `messages`).
GET	`/health`	Verificação de vida.

Erros

Situação	HTTP
Regra de negócio violada (DomainException)	422
Pedido inexistente	404
Conflito de versão após 3 tentativas	409
Entrada ou nome de tópico inválido	400

```
curl -X POST http://localhost:5080/api/orders -H "Content-Type: application/json" \
  -d '{"customerRef":"CLI-1001","submit":true,"items":[{"sku":"LIV-ARQ-
001","quantity":2,"unitPrice":129.90}]}'
```


Copyright (©) 2026 Clara Portes

08 Painel de monitoramento

Interface leve em HTML, CSS e JavaScript puros, alimentada por Server-Sent Events.

Área	O que mostra
Indicadores	Entrada e saída em msg/s, conexões, eventos gravados, outbox pendente e lag total, com mini-gráficos dos últimos 60 s.
Taxa de transferência	Publicadas × entregues nos últimos 2 minutos, com rolagem contínua e escala que se ajusta suavemente.
Filas e grupos	Cada tópico com offset e retenção; cada grupo com lag, mensagens em voo e membros. DLQs aparecem destacadas.
Conexões	Clientes TCP com papel (publisher, subscriber, ambos), contadores e fila de saída.
Event store	Eventos por tipo e totais de streams.
Sagas e outbox	Distribuição dos estados da saga e situação do outbox (pendentes, entregues, reagendadas, última falha).
Eventos recentes	Cauda do log global, com animação de entrada.
Replay	Escolha um stream e reproduza o estado versão a versão (play, avançar, voltar).

- **Simular tráfego** gera 30 pedidos (que percorrem a saga) e 1.500 mensagens de telemetria com consumidores de exemplo.
- **Reconstruir projeções** apaga o modelo de leitura e o reconstrói a partir do log.
- Tema claro/escuro segue o sistema e pode ser alternado no botão do topo.

Copyright (©) 2026 Clara Portes

09 Event Sourcing e CQRS

O estado do pedido nunca é gravado: ele é derivado da sequência imutável de eventos do stream.

Agregado Order

Comando	Exige	Evento
`CreateOrder`	cliente e moeda válidos	`OrderCreated`
`AddOrderItem`	status Draft	`OrderItemAdded`
`SubmitOrder`	Draft com itens	`OrderSubmitted`
`RecordPayment`	Submitted e valor = total	`PaymentProcessed`
`RecordPaymentFailure`	Submitted	`PaymentFailed`
`ReserveStock`	Paid	`StockReserved`
`RecordStockFailure`	Paid	`StockReservationFailed`
`RefundPayment`	pago e não estornado	`PaymentRefunded`
`ShipOrder`	StockReserved	`OrderShipped`
`CancelOrder`	não enviado; estornado se pago	`OrderCancelled`

Escrita

1. O `CommandDispatcher` resolve o handler por DI e repete até 3 vezes em `ConcurrencyException`.
2. O repositório carrega o snapshot mais recente e os eventos posteriores.
3. O agregado valida e gera eventos; o repositório grava eventos e outbox em uma transação com versão esperada.
4. A cada `SnapshotEvery` eventos, um snapshot é salvo.

Leitura

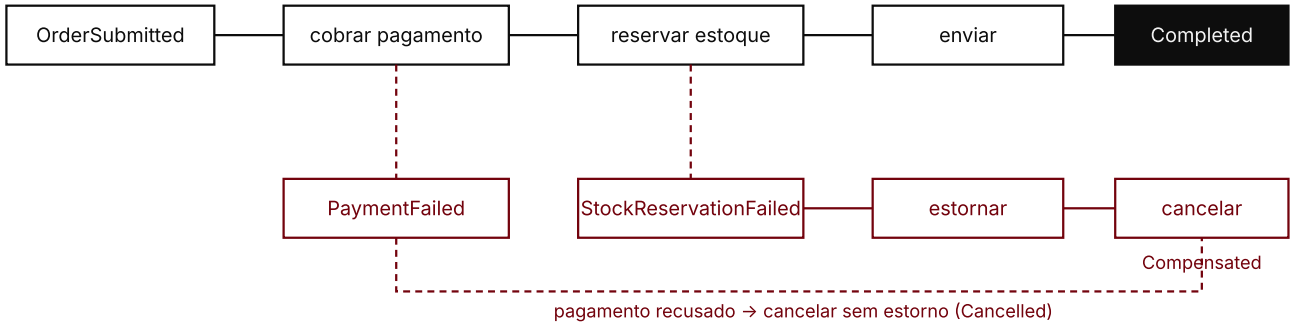
O `ProjectionWorker` lê o log global depois do checkpoint, aplica as projeções (`OrderSummaryProjection`, `DailySalesProjection`) e grava visões e checkpoint na mesma transação — cada evento é projetado exatamente uma vez. O worker acorda imediatamente após cada append do processo e, sem sinal, a cada segundo.

> **Reconstrução.** Como o modelo de leitura é derivado, ele pode ser apagado a qualquer momento: `POST /api/projections/rebuild` zera as visões e o replay recria tudo.

Copyright (©) 2026 Clara Portes

10 Saga de atendimento

Saga orquestrada que coordena pagamento, estoque e envio sem transação distribuída, compensando falhas.



Caminho feliz em preto; falhas e compensação em vermelho.

Evento recebido	Estado da saga	Ação
OrderSubmitted	AwaitingPayment	Cobra no gateway; envia RecordPayment ou RecordPaymentFailure
PaymentProcessed	ReservingStock	Reserva no estoque; envia ReserveStock ou RecordStockFailure
StockReserved	Shipping	Gera rastreio e envia ShipOrder
OrderShipped	Completed	Fim
PaymentFailed	Cancelled	Envia CancelOrder
StockReservationFailed	Compensating	Estorna no gateway e envia RefundPayment
PaymentRefunded	Compensating	Envia CancelOrder
OrderCancelled	Compensated ou Cancelled	Fim

A saga consome `orders.events` no grupo `saga.order-fulfillment`. Antes de tratar uma mensagem consulta `processed_messages`; depois de tratar, registra o id. Comandos repetidos são barrados pelas invariantes do agregado e os serviços simulados são idempotentes por pedido.

Copyright (©) 2026 Clara Portes

11 Outbox e garantias de entrega

Evento e mensagem nascem na mesma transação; a entrega ao broker é assíncrona, com retentativa e sem perda.

---

1. **Gravação atômica.** `SqlEventStore.AppendAsync` insere eventos e linhas do `outbox` na mesma transação.
2. **Reserva com lease.** O dispatcher marca até `BatchSize` mensagens com `locked_until`; no PostgreSQL usa `FOR UPDATE SKIP LOCKED`, permitindo vários servidores.
3. **Publicação em ordem.** As mensagens do lote saem na ordem do `id`.
4. **Confirmação.** Sucessos recebem `dispatched_at`; falhas incrementam `attempts` e agendam `next_attempt_at` com backoff exponencial e jitter.
5. **Limpeza.** Entregues com mais de `DispatchedRetention` são removidos a cada hora.

Garantia	Como é obtida
Sem perda	A mensagem só sai do outbox depois de publicada; falha de rede apenas reagenda.
At-least-once	Queda entre publicar e confirmar gera reenvio; consumidores deduplicam pelo `messageId`.
Sem dual-write	Não existe janela em que o evento foi gravado e a mensagem não.
Ordem por stream	Lotes sequenciais por `id`; consumidores podem validar `streamVersion`.

Copyright (©) 2026 Clara Portes

12 Testes e benchmarks

Suítes com xUnit v3, Testcontainers com PostgreSQL real e BenchmarkDotNet.

```
dotnet test tests/Brado.UnitTests
dotnet test tests/Brado.IntegrationTests
dotnet run -c Release --project benchmarks/Brado.Benchmarks -- --filter "*"
```

Suíte	Casos principais
ProtocolTests	Ida e volta do codec, tamanho, magic inválido, payload acima do limite, leitura sequencial, SUBSCRIBE, nomes de tópico.
OrderTests	Caminho feliz, invariantes, compensação, reidratação e snapshot.
BrokerTests	Offsets, competição entre membros, grupos independentes, prefetch, commit contíguo, NACK, DLQ, troca de membro, timeout, posição inicial.
SqliteEventStoreTests	Append/leitura, concorrência, imutabilidade, outbox atômico e reagendado, repositório com snapshot.
CqrsAndSagaTests	Saga com sucesso, compensação e falha de pagamento; projeção e rebuild; backoff.
BrokerTcpTests	Pub/sub real em TCP, offsets, divisão de carga, reentrega, desconexão e erro de protocolo.
PostgresTests	Ordem global, 8 escritores concorrentes, trigger, SKIP LOCKED com 4 workers, outbox sem perda, journal.

Benchmark	Mede
FrameCodecBenchmarks	Codificação em Span, leitura de cabeçalho e decodificação (64 B a 16 KiB).
BrokerBenchmarks	Publicação sem consumidores e publicação + consumo local.
ChannelBenchmarks	Channel limitado × ilimitado com leitor único.
EventStoreBenchmarks	Append com outbox no SQLite (1 e 10 eventos).

O cliente de demonstração também mede vazão ponta a ponta: `dotnet run --project samples/Brado.Demo -- bench --messages 200000 --size 256`.

Copyright (©) 2026 Clara Portes

13 Operação e solução de problemas

Recomendações para produção e respostas para os sintomas mais comuns.

Produção

- Use PostgreSQL para escrita e, de preferência, um banco separado para leitura.
- Coloque o broker em rede interna; para expor, use um proxy TCP com TLS na frente da porta 7420.
- Rode vários servidores com `Messaging:Mode=Tcp` apontando para um broker central: o outbox com `SKIP LOCKED` distribui a entrega sem duplicar reservas.
- Backup do event store com `pg_dump` ou PITR; o modelo de leitura não precisa de backup (é reconstruível).
- Monitore `/health`, o lag dos grupos e o outbox pendente em `/api/broker` e `/api/outbox`.

Sintoma	Causa provável	O que fazer
Lag crescendo em um grupo	Consumidores lentos ou prefetch baixo	Aumente membros do grupo ou o prefetch; verifique erros no handler.
Mensagens indo para a DLQ	Handler falhando repetidamente	Inspecione `<tópico>.dlq` em `/api/broker/topics/.../messages` e corrija o consumidor.
Outbox pendente sem cair	Broker inacessível no modo Tcp	Veja `lastError` em `/api/outbox`; as mensagens serão entregues quando o broker voltar.
HTTP 409 em comandos	Muitas escritas no mesmo pedido	Esperado sob disputa; o cliente pode repetir a requisição.
Modelo de leitura desatualizado	Projeção com erro	Verifique os logs do ProjectionWorker e use `/api/projections/rebuild`.
`database is locked` no SQLite	Escrita concorrente intensa	Migre para PostgreSQL; o SQLite já usa WAL e BEGIN IMMEDIATE.

Copyright (©) 2026 Clara Portes

14 Automação de commits

Workflow do GitHub Actions que registra um checkpoint por execução, cinco vezes ao dia.

Execução	Horário de Brasília	Cron (UTC)
1	09:17	`17 12 * * *`
2	11:43	`43 14 * * *`
3	14:08	`8 17 * * *`
4	16:52	`52 19 * * *`
5	20:26	`26 23 * * *`

- Arquivo: `.github/workflows/daily-activity.yml`. Cada execução acrescenta uma linha em `.github/activity/AAAA-MM.md` e faz um commit.
- Autor do commit: o dono do repositório, com e-mail `noreply` do GitHub montado em tempo de execução — nenhum dado pessoal no código.
- Concorrência controlada (`concurrency`) e push com `pull --rebase` e até 3 tentativas.
- Commits feitos com `GITHUB_TOKEN` não disparam o CI; o CI também ignora `.github/activity/**`.
- Para executar na hora: aba **Actions** → **Registro diário** → **Run workflow**. Para pausar: **Disable workflow**.
- O GitHub pode atrasar execuções agendadas em horários de pico; os minutos fora da hora cheia reduzem esse efeito.

Copyright (©) 2026 Clara Portes